Where Does Batching Help? A Measurement Study of Sequential, Static and Continuous Batching in a Toy LLM Inference Server

A Z Hasnain Kabir
hasnainkabir120@gmail.com
Code: github.com/HasnainKabir-repos/Toy-Inference-Server

Abstract

LLM serving is the process of deploying large language models to handle inference requests efficiently, optimizing latency, throughput, and memory utilization. Batching is a technique that improves serving efficiently by processing multiple requests together, but its effectiveness depends on how requests are scheduled and how the Key-value (KV) cache is managed. To investigate these factors, I built an inference server from scratch around Qwen3-0.6B and implemented four serving strategies: sequential execution, static batching, continuous batching without a merged forward pass, and continuous batching with batched decoding. My results show that dynamic scheduling alone provides little throughput benefit without batched computation, while batched decoding substantially improves throughput. I also found that the relative performance of static and continuous batching depends on output-length variability, with continuous batching offering significant latency benefits for mixed-length workloads. The study provides a baseline for understanding LLM serving mechanism limited by a small model, a single CPU, and short-context workloads.

1 Introduction

Serving a decoder-only language model requires repeated forward passes to generate output tokens. Since each pass involves reading model weights and processing the active requests, executing multiple requests in a single forward pass can amortize computational costs and improve throughput. Therefore, batching is a central technique in LLM serving. However, its effectiveness depends not only on how computation is grouped but also on how requests are scheduled and how the batched key-value (KV) cache is maintained.

Two questions motivate this work. Firstly, how much of the performance improvement comes from batched computation itself, compared to scheduling mechanisms that dynamically admit new requests and retire completed ones? Secondly, what does it cost to maintain the batched KV cache that makes this possible?

Production systems address these challenges through techniques such as iteration-level scheduling in Orca [2] and paged KV-cache management in vLLM [1]. These systems demonstrate the importance of scheduling and memory management at scale, primarily on GPUs. This work investigates the underlying mechanisms at a smaller scale, using a CPU-based inference server in which each strategy can be inspected and evaluated independently. My goal is not to propose a new serving technique, but to isolate the effects of scheduling, batched computation, and cache management through controlled experiments.

I build an inference server from scratch around Qwen3-0.6B and implement four serving strategies: sequential execution, static batching, continuous batching without a merged forward pass, and continuous batching with batched decoding. This design allows me to distinguish the benefits of dynamic request scheduling from those of executing active requests together in a single forward pass.

I evaluate these strategies using identical seeded request workloads under both offline execution and open-loop Poisson arrivals. I consider uniform and mixed output lengths to examine how variable length of requests affect batching efficiency and latency. The experiments use repeated runs with confidence intervals and compare generated outputs against the sequential engine to check behavioral equivalence. I measure throughput, end-to-end latency, slot occupancy, and the overhead of merging and rebuilding the padded KV cache.

The results show that scheduling alone does not improve throughput. Under offline execution, continuous batching without a merged forward pass achieves about the same throughput as sequential execution (9.3 tokens/s). Batched decoding, however, increases throughput by 1.96× for mixed output lengths and 2.36× for uniform lengths. Static batching performs at least as well as continuous batching for uniform lengths, while continuous batching reduces median end-to-end latency by about threefold for mixed lengths and achieves roughly twice the sequential capacity. Finally, KV-cache merging and rebuilding account for 5–7% of execution time in

short-context experiments. The overhead at longer contexts remains untested.

These findings suggest that dynamic scheduling and batched computation should be evaluated separately when analyzing LLM serving performance. They also show that the preferred batching strategy depends on the workload, particularly the variability in request output lengths. The results should be interpreted within the scope of the evaluation: a single small model running on one CPU, short prompts, and a limited number of repeated runs.

This paper makes three contributions:

- A small server that implements four strategies, arranged so that scheduling can be separated from batched computation.
- A benchmark with identical request streams, open-loop arrivals that avoid coordinated omission, repeated runs with confidence intervals, and an output-equivalence check against a reference engine.
- Measurements of throughput, latency, slot occupancy and cache-management overhead, with explicit statements of what the data does and does not support.

The remainder of the paper is organized as follows. Section 3 describes the four serving strategies, and Section 4 presents the experimental methodology. Section 5 reports throughput, latency, occupancy, and cache-management overhead. Section 6 presents the output-equivalence results, followed by a discussion of the findings in Section 7 and limitations in Section 8.

2 Background

KV Cache

During autoregressive decoding, each new token attends to the keys and values of all earlier tokens. Caching them means each step processes only the newest token, at the price of memory that grows by one position per step for every request in flight.

Static Batching

A group of requests is padded to a common length and run together until the longest one finishes. Rows that finish early continue to consume compute, and every response is released only when the whole batch completes.

Continuous Batching

The scheduler admits and retires requests at every decode step, so a short request leaves as soon as it is done and a waiting request immediately takes its slot.

Batched decode with unequal histories

Requests in a batch have different cache lengths, so the cache must be left-padded to a common length, accompanied by an attention mask that hides the padding and by per-row position ids. Whenever batch membership changes, this padded cache has to be rebuilt or edited. That cost is what block-based (paged) KV memory is designed to remove.

3 Systems

All four systems share one model loader, greedy decoding, and one request abstraction, so they differ only in how work is scheduled and batched.

- **Sequential.** Requests are served one at a time in arrival order, each with its own KV cache.
- **Static.** Up to B waiting requests are left-padded and decoded together until the longest finishes; all responses are then released at once.
- **Continuous, unbatched.** Requests are admitted and retired at every step, with up to B active, but each active request still runs its own forward pass. This variant isolates the effect of scheduling from the effect of batched computation.
- **Continuous, batched.** As above, except that all active requests share a single forward pass over a merged, left-padded cache, with a per-row attention mask and position ids. Each request's own cache is retained so the merged cache can be rebuilt when membership changes.

The batch cap is B = 4 in every experiment reported here.

4 Methodology

Model and hardware. All experiments use Qwen3-0.6B in float16 on a single CPU machine, with PyTorch and Hugging Face Transformers and greedy decoding.

Workloads. Requests are drawn from a pool of 16 short prompts using a fixed seed, so every system sees the identical stream within a repeat. Two workloads are reported: uniform, in which every request asks for 48 new tokens, and mixed, in which output budgets come from 8, 16, 32, 64, 96, 128 with a skew toward short outputs. Each run contains 24 requests. Generation stops at the end-of-sequence token or at the budget, and throughput is computed from the tokens actually generated.

Arrival modes. In the offline mode all requests are present at time zero. In the open-loop mode requests arrive as a Poisson process at 1x, 2x and 3x the sequential engine's measured request capacity for that stream. Arrivals follow

their schedule whatever the server is doing, and latency is measured from the scheduled arrival time, so a slow server cannot reduce its own load. This avoids the measurement error known as coordinated omission.

Metrics. Throughput is the number of generated tokens divided by the makespan. Time to first token (TTFT) runs from a request's arrival to the end of its prefill, and end-to-end latency from arrival to completion. I also record mean slot occupancy and the fraction of wall time spent building and extracting batched caches. Throughput is reported as a mean over repeats with a 95 percent confidence interval from the Student-t distribution. Latency percentiles are pooled over all requests in all repeats, so they carry no interval, and the p99 figures rest on about 72 requests (3 repeats) or 120 requests (5 repeats).

Repeats and sessions. Each repeat uses a different seed, a warmup beforehand, and a shuffled run order. The sequential, static and unbatched systems were run in one session (3 repeats, seeds 0 to 2). The batched engine was run in a second session (5 repeats, seeds 0 to 4) together with the sequential engine. The four-system comparisons (Tables 1 and 3, and Figures 1 and 2) therefore use seeds 0 to 2 for every system, taking the batched engine from the second session. Open-loop rates are scaled to the sequential capacity measured in each session, so offered load differs slightly between sessions; sequential throughput was about 9.2 tok/s in both. Table 2 2, which compares sequential and batched execution, and Figure 3a come entirely from the second session.

Correctness check. For every request I compare the generated token sequence with the sequential engine's output for the same request. The same check was run on a tiny float32 model, where numerical differences are not expected.

5 Results

5.1 Offline throughput

Scheduling alone does not raise throughput, and batched decode does. The unbatched continuous engine matches sequential execution in every configuration with tight intervals, while the batched engine is clearly faster. Whether continuous batching beats static batching depends on the workload: on mixed lengths their intervals overlap at three repeats, so throughput alone does not separate them. Figure 1 shows the same comparison graphically.

On uniform lengths both batching strategies keep all four slots occupied (mean occupancy 4.0), and static batching is about 7 percent faster than continuous batching (22.56 versus 20.99 tok/s; the intervals barely overlap). Continuous batching spends 5.6 percent of its wall time building and extracting

Workload	Batching strategy
Mixed	Sequential: 9.27 ± 0.50
Mixed	Continuous, unbatched: 9.30 ± 0.14
Mixed	Static: 13.32 ± 6.33
Mixed	Continuous, batched: 17.82 ± 3.25
Uniform	Sequential: 9.27 ± 0.23
Uniform	Continuous, unbatched: 9.24 ± 0.28
Uniform	Static: 22.56 ± 1.25
Uniform	Continuous, batched: 20.99 ± 0.46

Table 1: Offline throughput in tokens/s (mean ± 95% CI, 3 repeats, batch size $B = 4$).

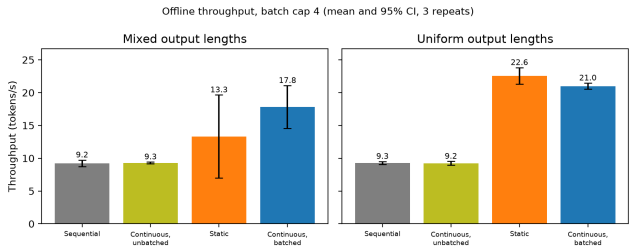


Figure 1: Offline throughput in tokens per second at batch cap 4, with one panel per workload (mixed output lengths on the left, uniform on the right). Bars show the mean over 3 repeats and error bars the 95 percent confidence interval. The sequential and unbatched bars are nearly identical, the batched continuous bar is roughly twice as tall, and the static bar has a wide interval on mixed lengths but is the tallest bar on uniform lengths.

Workload	Arrival	Sequential	Continuous, batched	Ratio
Mixed	Offline	9.12 ± 0.15	17.91 ± 1.19	1.96×
Mixed	1×	8.20 ± 0.62	9.89 ± 2.41	1.21×
Mixed	2×	8.97 ± 0.19	16.03 ± 1.55	1.79×
Mixed	3×	9.01 ± 0.15	17.11 ± 0.98	1.90×
Uniform	Offline	9.00 ± 0.44	21.21 ± 0.56	2.36×
Uniform	1×	8.40 ± 1.25	10.02 ± 2.96	1.19×
Uniform	2×	9.04 ± 0.27	16.97 ± 3.03	1.88×
Uniform	3×	9.14 ± 0.20	18.71 ± 1.79	2.05×

Table 2: Throughput in tok/s (mean ± 95% CI, 5 repeats). Offered load is relative to the sequential engine's measured capacity.

caches in this setting and static batching spends none, which is consistent with the gap, although I did not run an ablation to establish it.

5.2 Batched decode against sequential, five repeats

At 1x offered load the server is not saturated, so throughput mostly reflects the arrival rate and the ratio is small; la-

tency is the more informative metric there. Above 1x, the batched engine sustains roughly 1.8 to 2.1 times the sequential throughput, while sequential throughput saturates near 9 tok/s as expected. With 5 repeats the intervals separate the two engines in every offline row and in every 2x and 3x row.

5.3 Latency under load

On mixed lengths, continuous batching delivers about 3x lower median end-to-end latency than static batching at both 2x and 3x load (8.9 versus 27.6 seconds at 2x), and its median TTFT is far lower because freed slots are refilled immediately. On uniform lengths the two batching strategies are close in end-to-end latency (8.6 versus 10.9 seconds at 2x; 14.6 versus 14.5 at 3x), with continuous batching still ahead on TTFT. Scheduling alone, the unbatched engine, improves TTFT relative to sequential execution but not end-to-end latency or the tail, because the same work is merely interleaved. Figure 2 plots the medians against offered load. The p99 values in Table 3 3 are indicative only, since each rests on a few dozen requests.

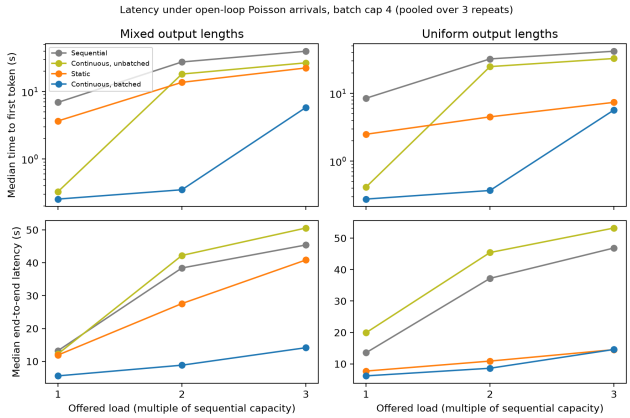


Figure 2: Median time to first token (top row, logarithmic axis) and median end-to-end latency (bottom row) against offered load, expressed as a multiple of the sequential engine's capacity, for the mixed (left) and uniform (right) workloads. Each line is one system at batch cap 4, pooled over 3 repeats. On mixed lengths the batched continuous line has the lowest end-to-end latency at every load. On uniform lengths it converges with static batching for end-to-end latency while staying lower for time to first token.

5.4 Slot occupancy and cache-management overhead

Static batching wastes slots when output lengths vary. On mixed lengths its mean occupancy is 2.1 of 4 slots, against 3.7 for continuous batching, because finished rows keep consuming compute until the longest request in the batch ends. On uniform lengths both strategies reach 4.0 (Figure 3b) 5.4.

The cost of keeping the batched cache consistent is modest at the short prompts used here. Building and extracting caches takes 6.5 percent of wall time offline on mixed lengths and 5.6 percent on uniform lengths (5 repeats). Under open-loop load the share grows with offered load, from about 0.5 to 0.9 percent at 1x, to 3.4 to 3.6 percent at 2x, and to 5.8 percent at 3x, because higher load keeps more requests in flight and changes batch membership more often (Figure 3a) 5.4.

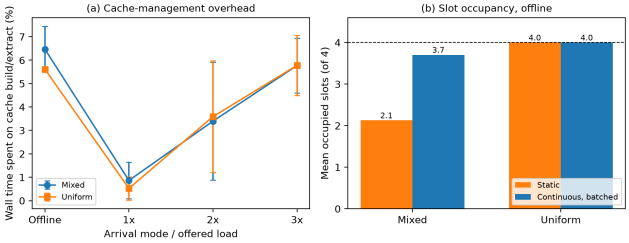


Figure 3. (a) Share of wall time that the continuous batched engine spends building and extracting batched caches, by arrival mode, for the two workloads (mean and 95 percent confidence interval over 5 repeats). (b) Mean number of occupied slots in the offline runs for static and continuous batching (3 repeats); the dashed line marks the cap of 4 slots. Panel (a) shows the overhead rising with load, and panel (b) shows static batching leaving about half its slots idle on mixed lengths.

6 Limitations

- The study covers one small model (0.6B parameters), one CPU, float16 and greedy decoding; results on GPUs, with larger models, or with sampling may differ.
- Prompts are short, so cache costs at long contexts were not measured.
- Samples are small: 24 requests per run, with 3 repeats for the four-system comparison and 5 for the sequential versus batched comparison. Several throughput intervals overlap, notably static versus continuous batching on mixed lengths, and latency percentiles are pooled across repeats without an interval.
- The batched engine and the other systems were run in different sessions. Seeds, workloads and machine were the same, but conditions were not identical, and offered loads differ slightly because they scale with each session's measured sequential capacity.
- Only one batch cap (B = 4) is reported.
- The float16 divergence interpretation (near-tied tokens) has not been verified directly.

Workload	Load	System	TTFT p50	E2E p50	E2E p99
Mixed	2×	Sequential	27.6	38.4	87.9
Mixed	2×	Continuous, unbatched	18.3	42.2	107.7
Mixed	2×	Static	13.7	27.6	53.9
Mixed	2×	Continuous, batched	0.35	8.9	39.9
Mixed	3×	Sequential	39.8	45.4	103.2
Mixed	3×	Continuous, unbatched	26.8	50.6	116.8
Mixed	3×	Static	22.5	40.9	64.0
Mixed	3×	Continuous, batched	5.8	14.2	56.9
Uniform	2×	Sequential	32.2	37.2	79.1
Uniform	2×	Continuous, unbatched	24.8	45.4	81.4
Uniform	2×	Static	4.5	10.9	21.3
Uniform	2×	Continuous, batched	0.37	8.6	22.4
Uniform	3×	Sequential	41.8	46.9	94.2
Uniform	3×	Continuous, unbatched	32.6	53.2	97.1
Uniform	3×	Static	7.4	14.5	28.0
Uniform	3×	Continuous, batched	5.7	14.6	30.7

Table 3: TTFT p50, end-to-end p50 and p99, in seconds (pooled over 3 repeats).

- Request streams are synthetic and drawn from a small prompt pool.

7 Related Work

Orca (Yu et al., 2022) [2] introduced iteration-level scheduling and selective batching for transformer serving, the idea behind my continuous engines. vLLM (Kwon et al., 2023) [1] introduced PagedAttention, which stores the KV cache in non-contiguous blocks to avoid the fragmentation and copying that a padded contiguous cache incurs. My open-loop load generation follows the standard remedy for coordinated omission in latency measurement. This study reproduces the qualitative behavior of those systems at toy scale, with the aim of isolating each mechanism rather than competing with them.

8 Reproducibility

The server, the benchmark harness, the analysis scripts and the raw per-request results (one JSON line per run) are in the project repository. Each results file begins with a metadata line recording library versions, thread count, the commit hash and the arguments used. The three figures are produced from the results files by a single script, described in the appendix.

References

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626. Association for Computing Machinery, 2023.

[2] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538. USENIX Association, 2022.